Book & Bloom — SE1020 Final Evaluation Guide

SLIIT — SE1020 Object Oriented Programming Prorata Project

A Complete Member-by-Member Technical Preparation & Viva Defense Manual

100/100 Marks Preparation | 6 Members Allocated | MySQL 8.0 & Atomic File Persistence | GitHub Verified

🏆 1. Official SE1020 Evaluation Rubric (100 Marks Breakdown)

Every single mark in the marking sheet is allocated to the following 7 criteria. Ensure all 6 members can demonstrate their part:

Assessment Criteria	Marks	What You Must Show Miss / Evaluators
Functionality of Assigned CRUD Operations	**30 Marks**	Complete Create, Read, Update, Delete for your assigned entity with live input validation and database updates.
Application of OOP Concepts	**20 Marks**	Encapsulation (private fields), Inheritance (super/subclasses), Polymorphism (dynamic dispatch), Abstraction, Information Hiding.
File Handling & Persistence Implementation	**10 Marks**	Safe atomic file persistence (`bookstore.txt`) and MySQL relational persistence (`book_bloom` schema).
User Interface Design & Usability	**10 Marks**	Clean, responsive web UI with minimum 3 UI states per module, modal editors, live search, and error banners.
Individual Contribution & GitHub Commits	**10 Marks**	Individual Git branch (`feature/...-IT...`) with commit history under your own SLIIT email address.
Presentation & Viva Performance	**10 Marks**	Confident technical walkthrough of code, answering theoretical OOP questions, and demonstrating rejected edge-cases.

Assessment Criteria	Marks	What You Must Show Miss / Evaluators
Documentation & Class Diagrams	**10 Marks**	Accurate UML Class Diagram, complete report, and documentation mapping to your actual Java classes.
TOTAL ALLOCATION	**100 Marks**	**Target: Grade A+ (Full 100/100)**

🌐 2. Core Architecture Questions (Asked from ANY Team Member)

Q1: Explain the High-Level Architecture of Book & Bloom.

Book & Bloom follows an Object-Oriented Three-Tier MVC Architecture: (1) **Presentation Tier:** JSP pages and responsive HTML5/CSS/JavaScript. (2) **Application / Business Tier:** An explicit Jakarta `ApiServlet` handles routing, session tokens, and CSRF protection before calling Domain Services (`UserService`, `BookService`, etc.). (3) **Persistence Tier:** Decoupled behind the `StoreRepository` interface supporting both atomic file storage (`bookstore.txt`) and MySQL relational storage (`MysqlStoreRepository`).

Q2: Why did you use Spring Boot with an explicit Jakarta Servlet together?

Spring Boot manages dependency injection, configuration, and the embedded Tomcat server. We wrote an explicit `ApiServlet` extending `HttpServlet` to demonstrate core Java Enterprise and Servlet fundamentals: manual HTTP method handling, Jackson JSON serialization, and explicit session / CSRF security lifecycle control.

Q3: Why is `BigDecimal` used for prices instead of `double` or `float`?

Binary floating-point types (`double`/`float`) cause precision loss due to base-2 representation (e.g. 0.1 + 0.2 = 0.30000000000000004). Financial systems require absolute precision, so Java's `BigDecimal` is used across all prices, subtotals, and delivery fee calculations.

Q4: How does Concurrency and Data Safety work?

Every mutation in `StoreRepository.transaction()` is thread-synchronized in memory. File operations use an OS `FileLock` (`store.lock`) and write to a `.tmp` file before executing an atomic move (`ATOMIC_MOVE`). In MySQL mode, relational transactions are managed under InnoDB with ACID transactions.

👥 3. Member-by-Member Viva Defense & Question Bank

Member 1: IT25102131 — User Management & Authentication

Branch: feature/user-management-IT25102131

Core Java Classes: `lk.bookbloom.model.User`, `Customer`, `AdminUser`, `UserService`, `Passwords`

Assigned CRUD Operations:

- **Create:** Customer account registration (`/register` ➔ `UserService.register()`).
- **Read:** Customer profile view (`/account`), Admin customer table (`/admin` ➔ Users tab).
- **Update:** Address, name, and contact details update (`UserService.updateProfile()`).
- **Delete:** Delete customer account with order constraint check (`UserService.delete()`).

3 UI States: (1) Registration page with live validation, (2) Profile editor at `/account`, (3) Admin user list with search & roles.

Demonstrate Rejected Action: Try registering with an existing email (e.g. `admin@bookbloom.lk`). The system blocks it: "*A customer with this email already exists.*"

Q: Where is Inheritance and Information Hiding in your module?

`Customer` and `AdminUser` both inherit from `abstract class User`, overriding `user.role()` polymorphically. Information hiding is enforced in `User.publicProfile()`, which strips out the sensitive `passwordHash` before JSON serialization.

Q: How are passwords secured?

Handled by `Passwords.java` using **PBKDF2WithHmacSHA256** with a secure 16-byte random salt and 210,000 iterations. Passwords are never stored in plaintext.

Member 2: IT25102114 — Book & Inventory Management

Branch: feature/book-management-IT25102114

Core Java Classes: `lk.bookbloom.model.Book`, `PrintedBook`, `EBook`, `BookService`

Assigned CRUD Operations:

- **Create:** Add Printed Book or Digital E-Book (`/admin` ➔ Books tab ➔ "+ Add Book").
- **Read:** Catalog search/filter (`/catalog`), Book detail page (`/book?id=...`), Admin inventory list.
- **Update:** Update book title, price, stock, description, format, and cover color.
- **Delete:** Delete title with active orders constraint protection.

3 UI States: (1) Catalog grid with search, (2) Book detail with interactive **"📖 Look Inside / Sample Read"** modal, (3) Admin book modal with dynamic Printed vs E-Book inputs.

Demonstrate Rejected Action: Try saving a book with a negative price (e.g. `-500`). `Rules.positive()` blocks it.

Q: Where is Polymorphism in the Book module?

In `Book.requiresShipping()`. `PrintedBook` overrides it to return `true` (requires physical courier delivery), while `EBook` overrides it to return `false`. `OrderService` calls `b.requiresShipping()` polymorphically to calculate delivery fees!

Member 3: IT25102130 — Order, Checkout & Delivery Management

Branch: feature/order-management-IT25102130

Core Java Classes: `lk.bookbloom.model.Order`, `OrderLine`, `OrderService`, `DeliveryFeePolicy`

Assigned CRUD Operations:

- **Create:** Shopping bag checkout (`/cart` ➔ `OrderService.create()`).
- **Read:** Customer order history (`/orders`), Admin dispatch dashboard (`/admin` ➔ Orders tab).
- **Update:** Lifecycle transitions: `PLACED` ➔ `PROCESSING` ➔ `COMPLETED`, or `CANCELLED`.
- **Delete:** Permanent deletion of cancelled orders (`OrderService.delete()`).

3 UI States: (1) Cart page with Free Delivery progress meter, (2) Order history with live courier tracking stepper, (3) Official Printable Tax Invoice with barcode.

Demonstrate Rejected Action: Try cancelling an already cancelled order. The state machine rejects it, guaranteeing stock is not restocked twice.

Q: How does atomic order cancellation restock inventory?

In `OrderService.updateStatus()`, when marked `CANCELLED`, each order line restores the book's stock: `b.setStock(b.getStock() + line.getQuantity())`. Both order state and inventory update inside the same synchronized transaction.

Member 4: IT25101966 — Author Management (Miss's Enhanced Module)

Branch: feature/author-management-IT25101966

Core Java Classes: `lk.bookbloom.model.Author`, `AuthorService`, `MysqlStoreRepository`

Assigned CRUD Operations:

- **Create:** Add Author with rich domain attributes (Name, Era, Nationality, Genre, Accolades, Photo URL, Bio).
- **Read:** Admin author table; Book detail "Author Spotlight" card; Catalog filter by author.
- **Update:** Edit biography, honors, nationality, primary genre, era, or photo URL.
- **Delete:** Safe author deletion with published books referential constraint.

3 UI States: (1) Admin 2-column responsive Author modal, (2) Admin Author table with era/nationality/awards crests, (3) Book Detail **"About the Author · ලේඛකයා පිළිබඳ විස්තරය" Spotlight Box**.

Demonstrate Rejected Action: Try deleting Martin Wickramasinghe. The system blocks it: "*Reassign or remove this author's books first.*"

Q: What improvements did you make based on Miss's feedback?

"Originally, the Author entity only had Name and Biography. Miss pointed out that this was too basic. In response, I expanded the `Author` OOP model with `nationality`, `primaryGenre`, `accolades` (literary awards), `bornYear` (active era), and `photoUrl`. I upgraded the Admin modal to a 2-column grid and built an 'About the Author' Spotlight Card on the book detail page."

Q: Where is Encapsulation in the Author model?

All attributes are declared `private`. Public getters and setters enforce null-safety (e.g. returning empty strings instead of null) and prevent direct mutable access from external classes.

Member 5: IT25102132 — Category Management & Navigation

Branch: feature/category-management-IT25102132

Core Java Classes: `lk.bookbloom.model.Category`, `CategoryService`

Assigned CRUD Operations:

- **Create:** Add new category (`/admin` ➔ Categories tab ➔ "+ Add Category").
- **Read:** Category cards on homepage, Category filter chips on catalog, Admin categories table.
- **Update:** Edit category name and description.
- **Delete:** Delete category with referential check (must contain 0 books).

3 UI States: (1) Popular Category cards on homepage, (2) Filter chips on catalog with active state, (3) Admin Category table with live titles counter.

Demonstrate Rejected Action: Try deleting the "Fiction" category. The system blocks it: "*Reassign or remove books in this category first.*"

Q: What is the relationship between Category and Book?

It is a **One-to-Many Association** (`Category 1 ——— * Book`). One category holds many books, but each book belongs to one category via `book.getCategoryId()`.

Member 6: IT25102519 — Review & Rating Management

Branch: feature/review-management-IT25102519

Core Java Classes: `lk.bookbloom.model.Review`, `ReviewService`

Assigned CRUD Operations:

- **Create:** Post verified review with 1-5 star rating (`/book?id=...` ➔ `ReviewService.submit()`).
- **Read:** Book reviews list, average star calculation, customer testimonials on homepage.
- **Update:** Edit existing review rating and comment by the review author.
- **Delete:** Customer deleting own review, or Administrator moderating inappropriate reviews.

3 UI States: (1) Interactive star rating selector on book page, (2) Verified reader review cards with dates, (3) Admin review moderation dashboard.

Demonstrate Rejected Action: Try submitting a second review for the same book from the same account. The system blocks it: "*You have already reviewed this title.*"

Q: How does Authorization work in the Review module?

In `ReviewService.delete()`, an Administrator can delete any review for moderation. Regular customers can only edit or delete reviews matching their own `userId` (`require(r.getUserId().equals(actor))`).

🎯 4. Live Presentation Walkthrough Sequence

1. **Start Server:** Double click `start-bookstore.cmd` on Desktop. Show Tomcat starting on port 8080 in under 5 seconds.
2. **Customer Flow:** Open `http://127.0.0.1:8080`. Show the cinematic hero, botanical banners, and popular categories.
3. **Interactive Sample Read:** Open *Gamperaliya* or *Atomic Habits* ➔ Click **"📖 Look Inside / Sample Read"** modal.
4. **Cart & Free Delivery:** Add books to bag to exceed LKR 5,000. Show the Free Delivery progress bar hit 100%. Click Checkout.
5. **Tax Invoice:** On order confirmation, click **"🖨️ Print Official Tax Invoice"** to show the printable receipt with barcode.
6. **Courier Stepper:** Open `/orders` to demonstrate live order tracking stepper.
7. **Admin Dashboard:** Log in as `admin@bookbloom.lk` / `BloomAdmin!2026`. Show Books, Categories, Orders, Customers, and Reviews tabs.
8. **Author Spotlight (Miss's Part):** Show Authors tab ➔ Open 2-column modal (show Era, Nationality, Genre, Accolades) ➔ Show Book Detail Author Spotlight box!
9. **Live MySQL Database Proof:** Open MySQL Workbench or CMD ➔ Run `USE book_bloom; SELECT * FROM authors;` to prove real-time persistence!
10. **GitHub Branches:** Open `https://github.com/IT25102131/book-and-bloom-se1020` ➔ Show the 6 student branches and commits.

> 💡 **Tip for Full Marks (Grade A+):** Stay calm, know your specific module flow (UI ➔ ApiServlet ➔ Service ➔ Repository ➔ MySQL), and demonstrate that one rejected edge-case to prove that your backend validates business rules properly!